

Compiladores: Panorama General y Contexto

Tipos de Lenguajes

M.C. MELISSA BRIGITTHE HINOJOSA CABELLO

Universidad Nacional Autónoma de México

Facultad de Ingeniería

25 de Agosto de 2026





A lo largo de la evolución de los lenguajes vimos que surgieron diversos lenguajes para atender propósitos específicos y resolver necesidades distintas, por ejemplo:

- ▶ Expresar algoritmos y lógica de control. → **Lenguajes de programación**
- ▶ Describir estructura y contenido de información. → **Lenguajes de marcado**
- ▶ Obtener información de grandes conjuntos de datos. → **Lenguajes de consulta**
- ▶ Automatizar tareas y coordinar otros sistemas. → **Lenguajes de scripting**

Entonces, si existen lenguajes con propósitos distintos,
¿todos se procesan de la misma manera?

Repaso: ¿Qué es un lenguaje de programación?

Un **lenguaje de programación** es un lenguaje formal diseñado para **describir mediante algoritmos** el comportamiento de un programa, controlar su flujo de ejecución, y definir **cómo debe resolverse un problema**.

Un lenguaje de programación comprende:

- ▶ Variables y tipos de datos.
- ▶ Funciones o procedimientos.
- ▶ Estructuras de control.
- ▶ Operaciones sobre datos.

```
public static int sumar(int a, int b) {  
    int resultado = a + b;  
  
    if (resultado >= 0) {  
        return resultado;  
    } else {  
        return 0;  
    }  
}
```



Un **lenguaje de marcado** es un lenguaje formal diseñado para **describir la estructura, organización o significado** de un documento. Es decir, describe **qué es** la información, **no qué hacer** con ella.

Los lenguajes de marcado se caracterizan por:

- ▶ Describir estructura y contenido.
- ▶ Organizar información mediante etiquetas.
- ▶ No incluir control de flujo ni lógica algorítmica.
- ▶ Ser procesados por herramientas de procesamiento de marcado.

```
<?xml version="1.0" encoding="UTF-8"?>
<persona>
  <nombre>Ana</nombre>
  <apellido>Barrera</apellido>
  <edad>21</edad>
  <carrera>Ingeniería en Computación</carrera>
  <correo>ana.barrera@comunidad.unam.mx</correo>
</persona>
```

Un **lenguaje de consulta** es un lenguaje formal diseñado para especificar **qué información se desea obtener** de un conjunto de datos, sin describir explícitamente cómo debe obtenerse. A diferencia del lenguaje de programación, un lenguaje de consulta indica el resultado esperado, mas no el algoritmo necesario para obtenerlo.

Los lenguajes de consulta se caracterizan por:

- ▶ Describir qué resultado se desea.
- ▶ Operar sobre conjuntos de datos estructurados o semi-estructurados.
- ▶ Tener una semántica declarativa.
- ▶ Ser procesados por motores de consulta de base de datos

```
SELECT d.nombre AS departamento,
       COUNT(e.id) AS total_empleados,
       AVG(e.salario) AS salario_promedio
FROM empleados e
JOIN departamentos d
    ON e.departamento_id = d.id
WHERE e.salario > 15000
GROUP BY d.nombre
HAVING COUNT(e.id) >= 5
ORDER BY salario_promedio DESC;
```

Lenguajes de scripting



Un **lenguaje de scripting** es un lenguaje diseñado para **automatizar** tareas, **coordinar** componentes existentes y **controlar la ejecución** de otros sistemas, priorizando rapidez y flexibilidad sobre eficiencia. Aunque tiene reglas sintácticas, no se define formalmente mediante gramáticas matemáticas completas (BNF/EBNF).

Los lenguajes de scripting se caracterizan por:

- ▶ Automatizar tareas repetitivas.
- ▶ Interactuar con otros programas o servicios.
- ▶ Tener tipado dinámico y sintaxis flexible.
- ▶ Ejecutarse típicamente mediante intérpretes.

```
#!/bin/bash

FECHA=$(date +"%Y-%m-%d")
ORIGEN="$HOME/documentos"
DESTINO="$HOME/backups/respaldo-$FECHA.tar.gz"

echo "Iniciando respaldo..."
tar -czf $DESTINO $ORIGEN

if [ $? -eq 0 ]; then
    echo "Respaldo completado correctamente."
else
    echo "Error durante el respaldo."
    exit 1
fi

echo "Proceso finalizado."
```

Lenguajes específicos de dominio



Cuando el problema es muy específico, diseñar un lenguaje propio puede ser más eficiente que usar uno general. Un **lenguaje específico de dominio** (DSL) es un lenguaje diseñado para **expresar soluciones dentro de un dominio** concreto, sacrificando generalidad a cambio de claridad y expresividad.

Los DSL se caracterizan por:

- ▶ Ser lenguajes especializados.
- ▶ Tener construcciones limitadas, pero muy expresivas.
- ▶ No buscan resolver cualquier problema.
- ▶ Su sintaxis es cercana al dominio que modelan.

```
\documentclass{article}

\title{Compiladores}
\author{Facultad de Ingeniería}
\date{\today}

\begin{document}
\maketitle

\section{Introducción}
\subsection{Antecedentes}
\subsection{Tipos de Lenguajes}
\end{document}
```

¿Cómo se procesan los lenguajes?



Hemos visto lenguajes muy distintos entre sí:

- ▶ Unos se consideran formales y otros no, aunque compartan características comunes.
- ▶ Unos describen procedimientos para obtener un resultado, otros solo la información.
- ▶ Unos permiten automatizar tareas y otros expresar soluciones específicas para un dominio.

Entonces, ¿todos los lenguajes se procesan de la misma forma?

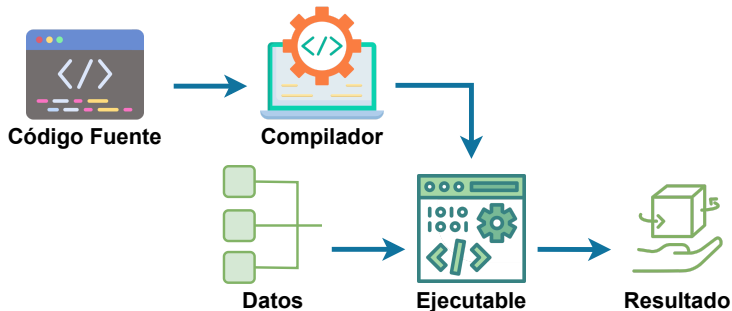
- Algunos lenguajes se traducen completamente antes de ejecutarse.
- Otros lenguajes se leen y ejecutan paso a paso, o se procesan directamente.
- En la práctica, muchos sistemas combinan traducción e interpretación.

Traducción completa



Parte de la idea fundamental de los compiladores:

- ▶ Un compilador toma todo el programa en **lenguaje fuente** y lo transforma a un **lenguaje destino** antes de que se ejecute.
- ▶ El resultado final de este proceso es un binario ejecutable diseñado para una **máquina de ejecución** específica, i.e., que depende directamente de la arquitectura del sistema.





¿Qué más sucede aquí?

- El binario resultante ya no necesita al compilador para ejecutarse.
- El programa en lenguaje destino suele ser más eficiente que el escrito en lenguaje fuente.
- Si existen, los errores se detectan antes de ejecutar el programa.
- Ciclo de desarrollo menos flexible: cada cambio, por pequeño que sea, requiere recompilar.



Hasta ahora hemos hablado de compiladores como sistemas que traducen programas. Un intérprete también es un **traductor**, pero con una diferencia clave en **cuándo y cómo** ocurre la ejecución.

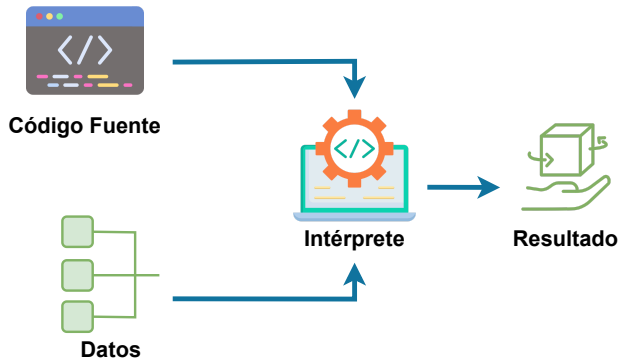
Intérprete

Es un sistema de software que toma un programa escrito en un lenguaje fuente y lo ejecuta directamente, realizando el análisis y la ejecución de sus instrucciones de manera progresiva sin generar un programa independiente en un lenguaje destino.

Lenguaje fuente y ejecución



A diferencia del compilador, con un intérprete **no existe una traducción completa** previa a un binario ejecutable. El lenguaje fuente sigue siendo el código que escribe el programador y el intérprete **actúa como máquina de ejecución**.



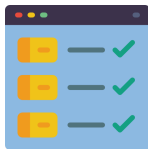
Lenguaje fuente y ejecución



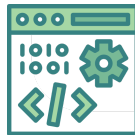
El programa puede empezar a ejecutarse aunque no esté completo dado que análisis y ejecución ocurren simultáneamente, por ello también es que los errores aparecen en tiempo de ejecución.



Análisis



Lectura



Ejecución

Este modelo hace que el ciclo de desarrollo sea:

- Más rápido.
- Más interactivo.
- Más adecuado para experimentación y scripting.



Los sistemas híbridos aparecen para abordar el compromiso entre ejecución flexible y eficiencia en tiempo de ejecución, **combinando ventajas** de los dos modelos anteriores. Al igual que en un compilador, el programa comienza como lenguaje fuente y pasa por una fase de traducción previa.

Sin embargo, el resultado no es código máquina nativo, sino una **representación intermedia**, comúnmente llamada **bytecode** que no depende directamente del hardware y está diseñado para ser ejecutado por una máquina virtual.

La máquina de ejecución es un runtime que:

- ▶ Interpreta el bytecode o lo compila dinámicamente durante la ejecución (como JIT).
- ▶ Gestiona memoria, seguridad y optimización.
- ▶ Permite mantener cierta flexibilidad durante el desarrollo y que un mismo programa pueda ejecutarse en distintas plataformas.

Comparación de modelos de ejecución



Modelo	Lenguaje destino	¿Cuándo se traduce?	¿Quién ejecuta?
Compilador	Código máquina	Antes de ejecutar	CPU
Intérprete	No hay binario	Durante la ejecución	Intérprete
Sistema híbrido	Bytecode / lenguaje intermedio	Antes y durante	Máquina virtual

Cuestionario diagnóstico: Autómatas





Tarea 2: Fases del Proceso de Compilación

→ Investigar acerca de las fases del proceso de compilación:

- Análisis léxico
- Análisis sintáctico
- Análisis semántico
- Representación intermedia
- Optimización

→ Elaborar un reporte con un resumen de los puntos anteriores (máximo 2 páginas).

→ Entrega: **Martes 1ro. de Septiembre** (antes de la clase).

- Enviar reporte (digital) por correo.
- Discusión en clase.